



ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL

Facultad de Ingeniería en Electricidad y Computación

SISTEMAS EMBEBIDOS

**Pong LED: Juego interactivo con comunicación entre
microcontroladores ATmega328P y PIC16F887**

DOCENTE

Ing. Solis Mesa Ronald David

Paralelo Teórico 1

Presentado por

Maquilón Castillo William, wmaquilo@espol.edu.ec

Vera Loor Zuleyka, zmvera@espol.edu.ec

Castillo Velasco David, dacavela@espol.edu.ec

Quiroz Vargas Saul, saquioz@espol.edu.ec

Fecha

09/06/2026

Contenido

1	Objetivo general	3
2	Objetivos específicos	3
3	Descripción técnica completa	3
3.1	Funcionamiento	3
3.2	Funciones de cada elemento del sistema	4
4	Capturas de la simulación en Proteus	5
5	Explicación del código para cada microcontrolador	8
5.1	ATMEGA328P	8
5.2	PIC16F887	23
6	Enlace del repositorio GitHub	27
7	Conclusiones	27
8	Recomendaciones	28

1 Objetivo general

Desarrollar un videojuego interactivo tipo Pong para un jugador mediante la integración de los microcontroladores ATmega328P y PIC16F887, utilizando una matriz LED 8x8 para representar el desplazamiento de la pelota y el movimiento de una raqueta, pulsadores para controlar la interacción del usuario y un buzzer para reproducir efectos sonoros sincronizados con los eventos del juego. El sistema deberá incorporar tres niveles de dificultad y ser simulado completamente en Proteus.

2 Objetivos específicos

- Diseñar e implementar la lógica de un videojuego tipo Pong para un jugador, en el cual una pelota rebote en tres paredes de la matriz LED y el usuario controle una raqueta para evitar que la pelota “caiga” por la parte derecha de la matriz del juego.
- Programar el microcontrolador ATmega328P para controlar la matriz LED 8x8, actualizar la posición de la pelota, representar visualmente la raqueta y detectar las colisiones y generar los rebotes durante la partida.
- Integrar pulsadores físicos conectados al ATmega328P para permitir el desplazamiento de la raqueta, la selección del nivel de dificultad y el reinicio del juego.
- Implementar tres niveles de dificultad mediante la modificación de la velocidad de desplazamiento de la pelota y, de ser necesario, el tamaño de la raqueta.
- Establecer una comunicación paralela entre el ATmega328P y el PIC16F887 para transmitir comandos asociados con los principales eventos del juego.
- Programar el microcontrolador PIC16F887 para interpretar los comandos recibidos desde el ATmega328P y reproducir melodías o efectos sonoros vinculados con el inicio de la partida, los rebotes, el cambio de nivel, la victoria y la derrota.
- Simular y validar el funcionamiento completo del sistema en Proteus, verificando la interacción entre los microcontroladores, la matriz LED 8x8, los pulsadores y el buzzer.

3 Descripción técnica completa

Este es el juego “Pong”, un juego de un solo jugador, programado en lenguaje C y simulado en el software Proteus. Está compuesto por varios elementos: cinco botones para que el jugador pueda interactuar con el sistema, una matriz led 8x8 que permite la visualización del juego y los microcontroladores ATMEGA328P y PIC16F887, que van a contener la lógica y los efectos de sonido correspondientes.

3.1 Funcionamiento

El juego inicia con un mensaje de bienvenida que se desplaza horizontalmente en la matriz de visualización y al mismo tiempo se ejecuta el sonido que indica que se ha inicializado el

juego; mientras el mensaje se mantenga desplazándose el sonido de bienvenida va a mantenerse.

Para que el sistema cambie de estado el jugador debe presionar el pulsador “SELECCIONAR NIVEL”, en la pantalla a cada pulsación se verá una letra: “F” para nivel fácil, “N” para nivel normal o intermedio y “D” para el nivel difícil; la dificultad para cada uno de estos niveles está dada por una velocidad, que aumenta con respecto al nivel que se elija (60 para “F”, 100 para “N” y 150 para “D”), en cada “desplazamiento” entre niveles, se ejecutará en paralelo una señal sonora. Para seleccionar uno de estos niveles basta con pulsar el botón “INICIAR PARTIDA”, de manera que se de inicio a la partida en el nivel seleccionado por el jugador.

Al iniciar la partida, se visualiza en la matriz el espacio del juego, en el que la barra que representa a la raqueta se encuentra en la parte derecha de la misma y la pelota que comenzará a desplazarse por la matriz, rebotando en las paredes (borde superior, inferior e izquierdo de la matriz); el “rebote” consiste en que al chocar con las paredes la pelota debe visualizarse “cambiando” de dirección, también existe una alerta sonora de rebote.

Aquí es cuando el jugador toma la decisión sobre a qué lugar quiere desplazar la “barra” raqueta y puede pulsar los botones de “SUBIR” o “ABAJO”, para desplazarse verticalmente sobre el borde, incrementando o decrementando su posición.

La pelota se seguirá desplazando hasta que la trayectoria de la pelota se acerque al borde derecho, en el que existen dos alternativas.

1. La pelota llega al borde derecho y su posición coincide con uno de los dos bits de la raqueta, entonces el jugador suma un punto a lo acumulado y la pelota “rebota”, vuelve a desplazarse por la matriz y se emite un sonido de que ha habido un rebote exitoso y continua el juego.

2. La pelota llega al borde derecho y su posición no coincide con ninguno de los dos bits de la raqueta, en ese caso el jugador pierde la partida y se emite un sonido de derrota, en la matriz aparece el puntaje acumulado por el jugador.

Si el jugador logra acumular 10 puntos en la partida, se termina el juego y aparece en la matriz un mensaje que se desplaza con la palabra “GANADOR” tres veces consecutivas.

Luego de que se llega a una victoria o una derrota, la matriz se apaga y el sistema del juego se reinicia automáticamente, esperando una nueva ejecución.

3.2 Funciones de cada elemento del sistema

Botón “SUBIR”: Permite el desplazamiento de la barra (raqueta) de 2 bits hacia arriba de la columna.

Botón “ABAJO”: Permite el desplazamiento de la barra (raqueta) de 2 bits hacia abajo de la columna.

Botón “SELECCIONAR NIVEL”: Permite desplazarse entre los 3 niveles de dificultad.

Botón “INICIAR PARTIDA”: Permite seleccionar uno de los tres niveles de dificultad.

Botón “RESETEAR”: Permite reiniciar ambos microcontroladores.

Matriz led 8x8: Permite la visualización del sistema del juego, actúa como interfaz gráfica.

Buzzer: Permite reproducir los sonidos del sistema para que mejorar la experiencia de usuario.

Microcontrolador ATMEGA328P: Es el cerebro principal del juego, aquí se encuentra almacenada la lógica de este y la interacción con los elementos físicos externos del sistema como la matriz y los botones, este tiene una conexión paralela con el PIC16F887.

Microcontrolador PIC16F887: Este es considerado el segundo cerebro del sistema, es usado para la interacción simultanea con el ATMEGA, permitiendo la reproducción de sonidos en paralelo a lo que sucede en el juego, sin que se necesite manejar interrupciones, este se conecta al buzzer, permitiendo la salida de sonido.

4 Capturas de la simulación en Proteus

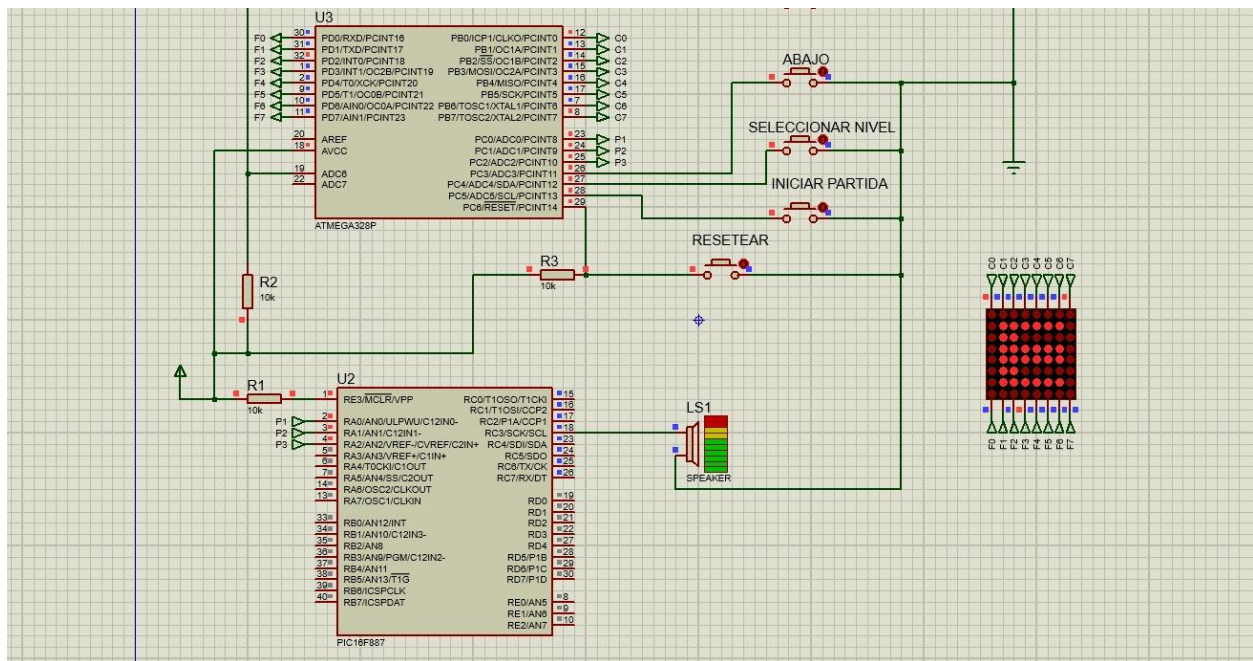


Fig1.Estado de inicio de juego, mensaje de bienvenida.

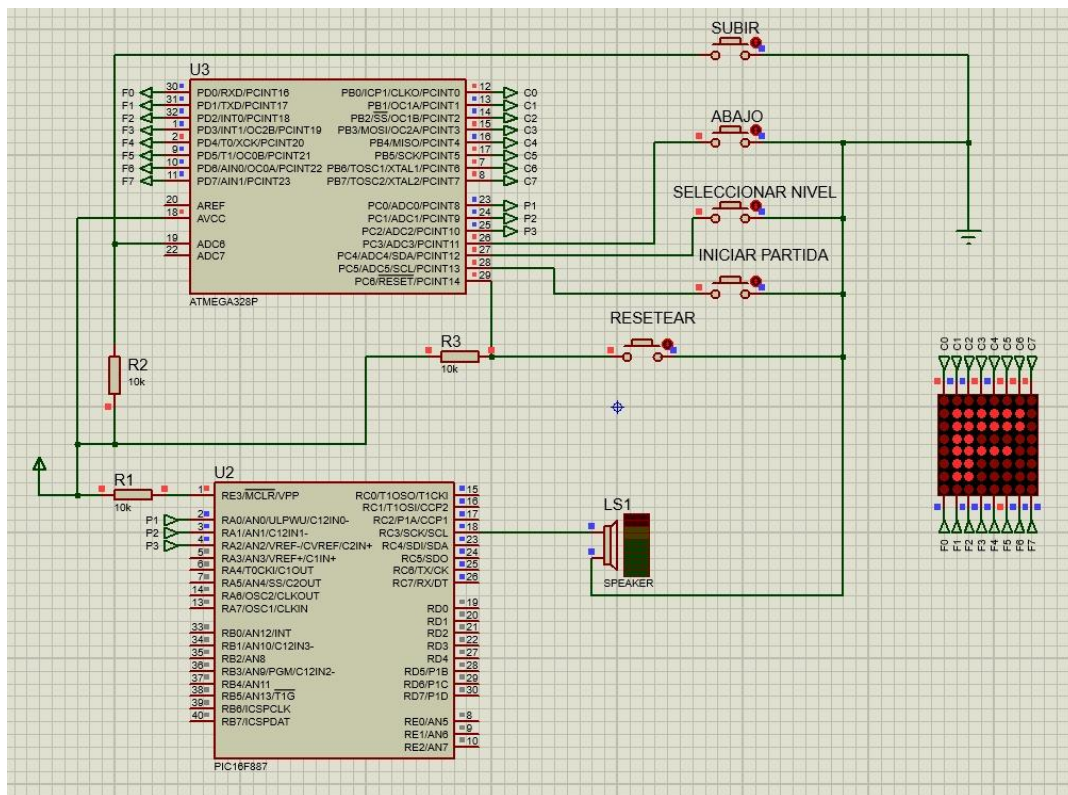


Fig2. Selección de nivel, nivel fácil.

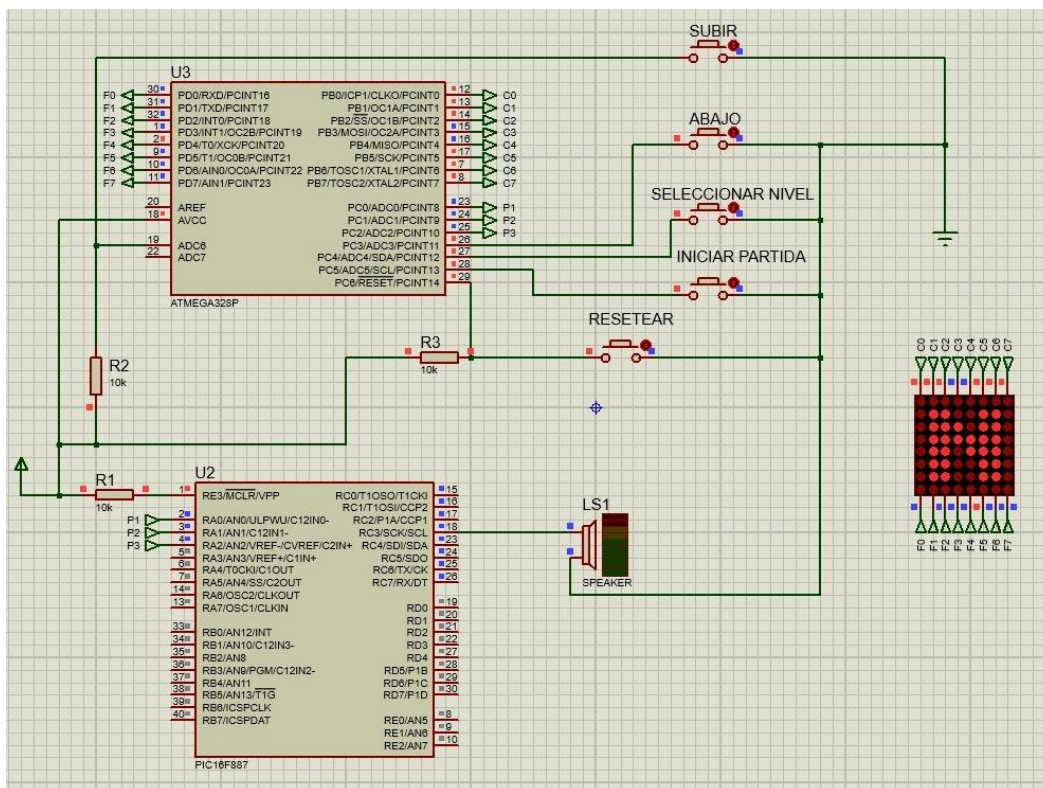


Fig3. Selección de nivel, nivel normal o intermedio.

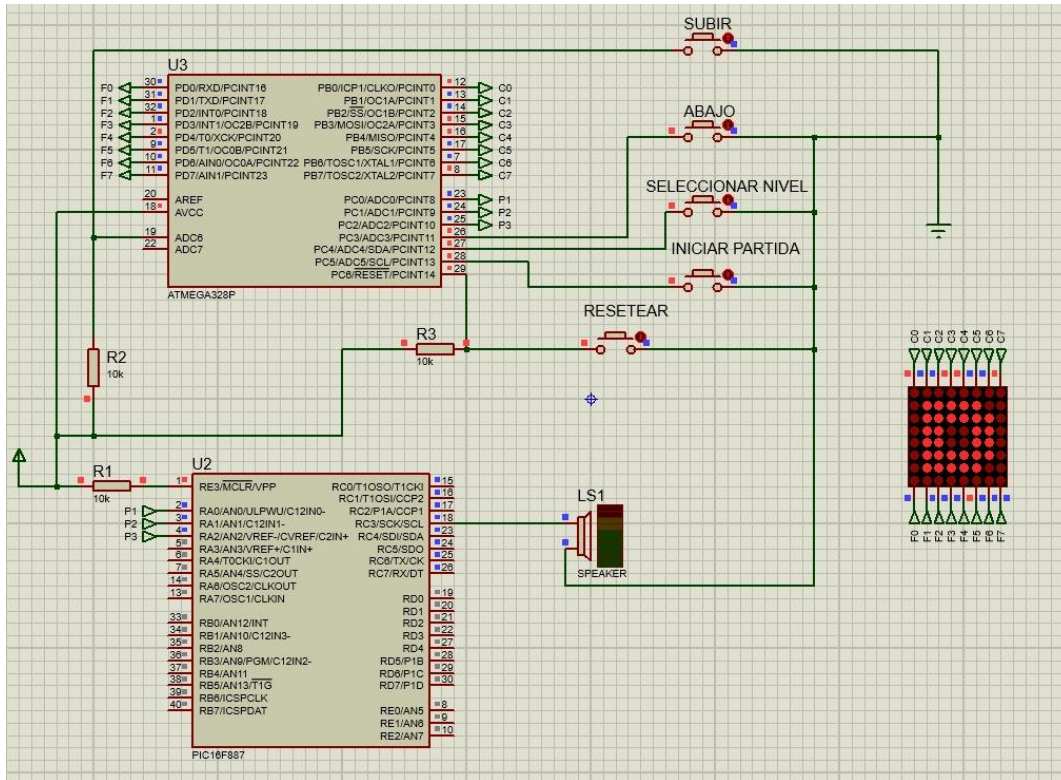


Fig4. Selección de nivel, nivel difícil.

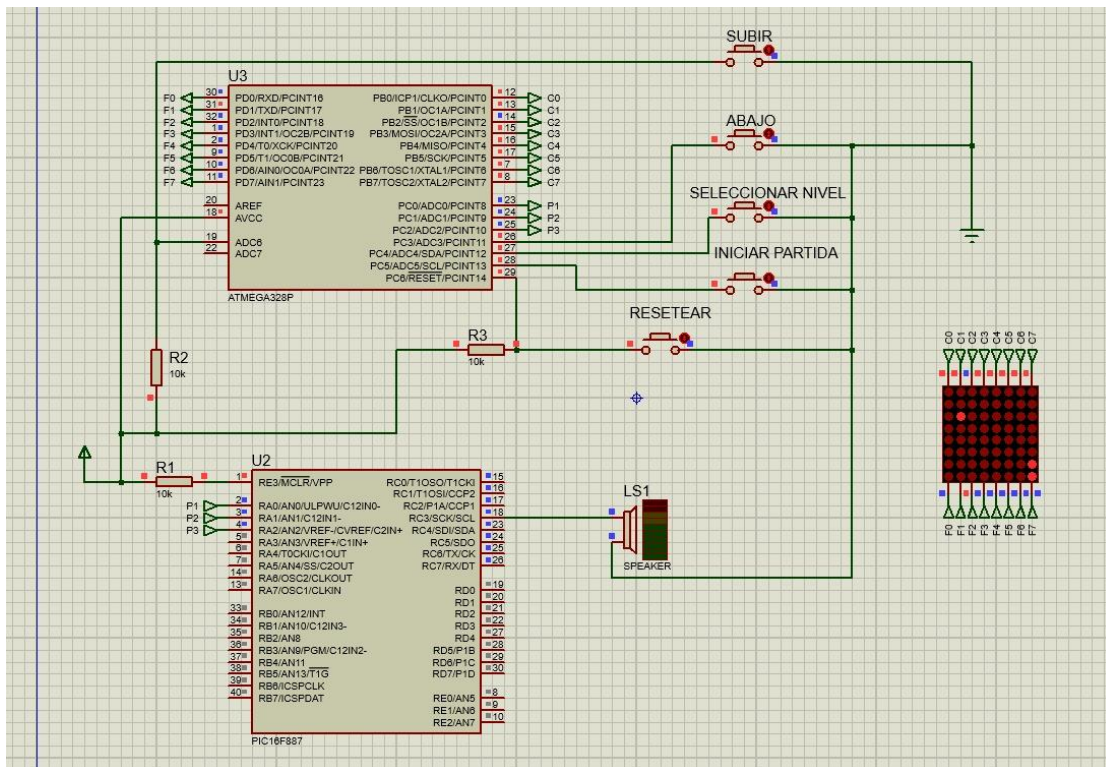


Fig5. Partida, bucle de juego.

5 Explicación del código para cada microcontrolador

5.1 ATMEGA328P

```
#define F_CPU 8000000UL

#include <avr/io.h>
#include <util/delay.h>
```

Se establece la frecuencia del microcontrolador en 8 MHz, se incluyen las librerías necesarias.

```
#define TAM_BIENVENIDA 96
#define TAM_GANADOR 72
```

Se define el tamaño de los mensajes desplazables en la matriz.

```
#define CMD_SILENCIO 0
#define CMD_INICIO 1
#define CMD_PARED 2
#define CMD_PALETA 3
#define CMD_NIVEL 4
#define CMD_DERROTA 5
#define CMD_VICTORIA 6
#define CMD_BIENVENIDA 7
```

Se definen los comandos de los estados del juego para poder enviarlos al PIC.

```
unsigned char BIENVENIDA[] = {
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,

    0x00, 0x7E, 0x7E, 0x5A, 0x5A, 0x7E, 0x3C, 0x00, // B
    0x00, 0x66, 0x66, 0x7E, 0x7E, 0x66, 0x66, 0x00, // I
    0x00, 0x7E, 0x7E, 0x5A, 0x5A, 0x5A, 0x5A, 0x00, // E
    0x00, 0x7E, 0x7E, 0x0C, 0x18, 0x7E, 0x7E, 0x00, // N
    0x00, 0x1E, 0x3E, 0x60, 0x60, 0x3E, 0x1E, 0x00, // V
    0x00, 0x7E, 0x7E, 0x5A, 0x5A, 0x5A, 0x5A, 0x00, // E
    0x00, 0x7E, 0x7E, 0x0C, 0x18, 0x7E, 0x7E, 0x00, // N
    0x00, 0x66, 0x66, 0x7E, 0x7E, 0x66, 0x66, 0x00, // I
    0x00, 0x7E, 0x7E, 0x66, 0x66, 0x7E, 0x3C, 0x00, // D
    0x00, 0x7E, 0x7E, 0x66, 0x66, 0x7E, 0x7E, 0x00, // O

    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
};
```


Definición de la matriz del mensaje de bienvenida.

```
unsigned char GANADOR[] = {
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,

    0x00, 0x7E, 0x7E, 0x46, 0x56, 0x76, 0x76, 0x00, // G
    0x00, 0x7E, 0x7E, 0x1A, 0x1A, 0x7E, 0x7E, 0x00, // A
    0x00, 0x7E, 0x7E, 0x0C, 0x18, 0x7E, 0x7E, 0x00, // N
    0x00, 0x7E, 0x7E, 0x1A, 0x1A, 0x7E, 0x7E, 0x00, // A
    0x00, 0x7E, 0x7E, 0x66, 0x66, 0x7E, 0x3C, 0x00, // D
    0x00, 0x7E, 0x7E, 0x66, 0x66, 0x7E, 0x7E, 0x00, // O
    0x00, 0x7E, 0x7E, 0x12, 0x12, 0x3E, 0x6C, 0x00, // R

    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
};
```

Definición del mensaje para la victoria.

```
unsigned char MODO_FACIL[] = {
    0x00, 0x7E, 0x7E, 0x16, 0x16, 0x16, 0x06, 0x00
};

unsigned char MODO_NORMAL[] = {
    0x00, 0x7E, 0x7E, 0x0C, 0x18, 0x7E, 0x7E, 0x00
};

unsigned char MODO_DIFICIL[] = {
    0x00, 0x7E, 0x7E, 0x66, 0x66, 0x7E, 0x3C, 0x00
};
```

Definición de arreglos de letras para representar niveles (F,N,D)

```
unsigned char DIGITO_0[] = {
    0x00, 0x7E, 0x7E, 0x66, 0x66, 0x7E, 0x7E, 0x00
};

unsigned char DIGITO_1[] = {
    0x00, 0x08, 0x04, 0x7E, 0x7E, 0x00, 0x00, 0x00
};

unsigned char DIGITO_2[] = {
    0x00, 0x66, 0x76, 0x7E, 0x5E, 0x4C, 0x40, 0x00
};

unsigned char DIGITO_3[] = {
    0x00, 0x5A, 0x5A, 0x5A, 0x5A, 0x5E, 0x7E, 0x00
};
```

```

};

unsigned char DIGITO_4[] = {
    0x00, 0x0E, 0x0E, 0x08, 0x08, 0x7E, 0x7E, 0x00
};

unsigned char DIGITO_5[] = {
    0x00, 0x5E, 0x5E, 0x5E, 0x76, 0x76, 0x76, 0x00
};

unsigned char DIGITO_6[] = {
    0x00, 0x7E, 0x7E, 0x4A, 0x4A, 0x7A, 0x7A, 0x00
};

unsigned char DIGITO_7[] = {
    0x00, 0x46, 0x66, 0x36, 0x1E, 0x0E, 0x06, 0x00
};

unsigned char DIGITO_8[] = {
    0x00, 0x7E, 0x7E, 0x56, 0x56, 0x7E, 0x7E, 0x00
};

unsigned char DIGITO_9[] = {
    0x00, 0x1E, 0x1E, 0x16, 0x76, 0x76, 0x7E, 0x00
};

```

Definición de arreglos de números para representar puntaje.

```

unsigned char PUNTOS[] = {
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,

    0x00, 0x7E, 0x7E, 0x12, 0x12, 0x1E, 0x0C, 0x00, // P
    0x00, 0x7E, 0x7E, 0x60, 0x60, 0x7E, 0x7E, 0x00, // U
    0x00, 0x7E, 0x7E, 0x0C, 0x18, 0x7E, 0x7E, 0x00, // N
    0x00, 0x06, 0x06, 0x7E, 0x7E, 0x06, 0x06, 0x00, // T
    0x00, 0x7E, 0x7E, 0x66, 0x66, 0x7E, 0x7E, 0x00, // O
    0x00, 0x5E, 0x5E, 0x56, 0x76, 0x76, 0x76, 0x00, // S

    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00
};

```

Definición del arreglo de la palabra “puntos”.

```

unsigned char posiciones[8] = {
    1, 2, 4, 8, 16, 32, 64, 128
};

```

Arreglo de números que representan filas.

```

void enviar_comando_pic(unsigned char comando) {
    // Conservar el estado de PC3-PC5 y modificar PC0-PC2.
    PORTC =
        (PORTC & 0xF8) |
        (comando & 0x07);

    _delay_ms(20);

    // Regresar a silencio.
    PORTC &= 0xF8;
}

```

Esta función envía un comando por el puerto C hacia el PIC, mantiene el comando por 20 ms y luego lo regresa al estado de silencio.

```

void iniciar_musica_bienvenida() {
    PORTC =
        (PORTC & 0xF8) |
        CMD_BIENVENIDA;
}

void detener_musica_bienvenida() {
    PORTC &= 0xF8;
}

```

Estas dos funciones en conjunto permiten que se reproduzca la melodía de bienvenida enviando el respectivo comando y la otra regresa el puerto C al silencio

```

void inicializar_adc() {
    // Usar AVCC como referencia y seleccionar ADC6.
    ADMUX =
        (1 << REFS0) |
        6;

    // Activar ADC con prescaler de 64.
    // Frecuencia del ADC: 8 MHz / 64 = 125 kHz.
    ADCSRA =
        (1 << ADEN) |
        (1 << ADPS2) |
        (1 << ADPS1);
}

```

Esta función lo que hace es convertir un pin analógico para que se comporte como uno digital.

```

unsigned int leer_adc6() {
    // Seleccionar ADC6 conservando la referencia AVCC.
    ADMUX =
        (ADMUX & 0xF0) |
        6;
}

```

```

// Iniciar conversión.
ADCSRA |=
    (1 << ADSC);

// Esperar hasta finalizar la conversión.
while (ADCSRA & (1 << ADSC)) {
}

return ADC;
}

```

Esta función permite leer un pin analógico.

```

unsigned char boton_subir_presionado() {
    if (leer_adc6() < 200) {
        return 1;
    }

    return 0;
}

```

Esta función me permite saber si mi botón fue presionado, comparando la lectura con un umbral.

```

void estado_bienvenido() {
    // Iniciar la música antes de mostrar el mensaje.
    iniciar_musica_bienvenida();

    while (1) {
        for (
            int i = 0;
            i <= TAM_BIENVENIDA - 8;
            i++
        ) {
            for (int k = 0; k < 50; k++) {
                for (int j = 0; j < 8; j++) {
                    // Activar una fila y mostrar sus columnas.
                    PORTD =
                        posiciones[j];

                    PORTB =
                        ~BIENVENIDA[i + j];

                    _delay_ms(0.05);
                }

                // PC4 permite salir del mensaje de bienvenida.
                if (!(PINC & (1 << PC4))) {

```

Esta función permite desplazar el mensaje de bienvenida, lo primero que hace es llamar a la función que le manda un comando al PIC para que reproduzca la melodía de bienvenida, luego lo que hace es realizar la respectiva lógica de desplazamiento y establece que si se presiona el botón de nivel se deja de presentar el mensaje y manda el comando para que el PIC se silencie.

13


```

    if (!(PINC & (1 << PC4))) {
        nivel =
            (nivel + 1) % 3;

        enviar_comando_pic(
            CMD_NIVEL
        );

        _delay_ms(200);
    }

    // Confirmar nivel y continuar con el juego.
    if (!(PINC & (1 << PC5))) {
        PORTD = 0x00;
        PORTB = 0xFF;

        _delay_ms(200);

        return nivel;
    }
}

```

Esta función establece las condiciones para que se pueda cambiar entre niveles, mientras se envía el comando al PIC para que se reproduzca el sonido correspondiente al comando, luego permite que al ser presionado un botón se seleccione el nivel, esta función me retorna el nivel.

```

int estado_juego(int nivel) {
    int velocidad;

    // Ajustar velocidad según la dificultad elegida.
    if (nivel == 0) {
        velocidad = 150;
    }
    else if (nivel == 1) {
        velocidad = 100;
    }
    else {
        velocidad = 60;
    }

    // Posición inicial de la paleta.
    int columna_paleta =
        3;

    // Posición inicial de la pelota.

```

```

int fila_bola =
    4;

int columna_bola =
    3;

// Dirección inicial de la pelota.
int direccion_fila =
    1;

int direccion_columna =
    1;

// Puntaje acumulado.
int puntos =
    0;

// Contadores utilizados para controlar la velocidad.
int contador_velocidad =
    0;

int contador_paleta =
    0;

// Avisar al PIC que la partida comenzó.
enviar_comando_pic(
    CMD_INICIO
);

while (1) {
    // Crear una imagen temporal del juego.
    unsigned char frame[8] = {
        0x00, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x00
    };

    // Dibujar la paleta con dos LED consecutivos.
    frame[7] |=
        (1 << columna_paleta);

    frame[7] |=
        (1 << (columna_paleta + 1));

    // Dibujar la pelota.
    frame[fila_bola] |=

```

```

        (1 << columna_bola);

// Mostrar el frame mediante multiplexación.
for (int j = 0; j < 8; j++) {
    PORTD =
        posiciones[j];

    PORTB =
        ~frame[j];

    _delay_ms(0.05);
}

// -----
// CONTROLAR EL MOVIMIENTO DE LA PALETA
// -----

contador_paleta++;

if (contador_paleta >= 20) {
    contador_paleta = 0;

    // ADC6 reemplaza al pin PC2 original.
    // Permite desplazar la paleta en una dirección.
    if (boton_subir_presionado()) {
        if (columna_paleta > 0) {
            columna_paleta--;
        }
    }

    // PC3 desplaza la paleta en la dirección contraria.
    if (!(PINC & (1 << PC3))) {
        if (columna_paleta < 6) {
            columna_paleta++;
        }
    }
}

// -----
// ACTUALIZAR LA POSICIÓN DE LA PELOTA
// -----

contador_velocidad++;

if (contador_velocidad >= velocidad) {

```

```

contador_velocidad = 0;

// Calcular la siguiente posición de la pelota.
int siguiente_fila =
    fila_bola +
    direccion_fila;

int siguiente_columna =
    columna_bola +
    direccion_columna;

// -----
// VERIFICAR CONTACTO CON LA PALETA
// -----

if (fila_bola == 7) {
    if (
        columna_bola == columna_paleta ||
        columna_bola == columna_paleta + 1
    ) {
        // Aumentar puntaje después de un rebote válido.
        puntos++;

        // Finalizar la partida al alcanzar 10 puntos.
        if (puntos >= 10) {
            enviar_comando_pic(
                CMD_VICTORIA
            );

            return -1;
        }

        // Reproducir sonido de rebote contra paleta.
        enviar_comando_pic(
            CMD_PALETA
        );

        // Invertir dirección vertical.
        direccion_fila =
            -direccion_fila;

        siguiente_fila =
            fila_bola +
            direccion_fila;
    }
}

```

```

        // Corregir trayectoria si también alcanza un borde.
        if (
            siguiente_columna <= 0 ||
            siguiente_columna >= 7
        ) {
            direccion_columna =
                -direccion_columna;

            siguiente_columna =
                columna_bola +
                direccion_columna;
        }
    }
    else {
        // La pelota no coincidió con la paleta.
        enviar_comando_pic(
            CMD_DERROTA
        );

        return puntos;
    }
}

// -----
// REBOTE CONTRA LA PARED SUPERIOR
// -----

else if (siguiente_fila <= 0) {
    direccion_fila =
        -direccion_fila;

    siguiente_fila =
        fila_bola +
        direccion_fila;

    enviar_comando_pic(
        CMD_PARED
    );

    // Corregir trayectoria si alcanza una esquina.
    if (
        siguiente_columna <= 0 ||
        siguiente_columna >= 7
    ) {
        direccion_columna =

```

```

        -direccion_columna;

        siguiente_columna =
            columna_bola +
            direccion_columna;
    }
}

// -----
// REBOTE CONTRA PAREDES LATERALES
// -----

else if (
    siguiente_columna <= 0 ||
    siguiente_columna >= 7
) {
    direccion_columna =
        -direccion_columna;

    siguiente_columna =
        columna_bola +
        direccion_columna;

    enviar_comando_pic(
        CMD_PARED
    );
}

// Guardar la nueva posición de la pelota.
fila_bola =
    siguiente_fila;

columna_bola =
    siguiente_columna;
}
}
}

```

Esta función es la lógica base del juego, lo primero que hace es setear la velocidad del juego de acuerdo con el nivel, de ahí efectúa la lógica de desplazamientos, rebotes. Activa y desactiva los comandos que se envían al PIC para que los sonidos se ejecuten según la ejecución del programa, además lee el estado de los pulsadores de dirección para modificar la posición de la barra. Esta función me retorna un valor de estado del juego.

```
void estado_ganador() {
```

```

for (int rep = 0; rep < 3; rep++) {
    for (
        int i = 0;
        i <= TAM_GANADOR - 8;
        i++
    ) {
        for (int k = 0; k < 50; k++) {
            for (int j = 0; j < 8; j++) {
                PORTD =
                    posiciones[j];

                PORTB =
                    ~GANADOR[i + j];

                _delay_ms(0.05);
            }
        }
    }

    // Apagar matriz después del mensaje.
    PORTD = 0x00;
    PORTB = 0xFF;
}

```

Esta función lo que hace es desplazar tres veces horizontalmente el mensaje de victoria.

```

void estado_puntuacion(int puntos) {
    int decenas =
        puntos / 10;

    int unidades =
        puntos % 10;

    // Asociar cada número con su matriz correspondiente.
    unsigned char *digitos[10] = {
        DIGITO_0,
        DIGITO_1,
        DIGITO_2,
        DIGITO_3,
        DIGITO_4,
        DIGITO_5,
        DIGITO_6,
        DIGITO_7,
        DIGITO_8,
        DIGITO_9
    }
}

```



```

};

// Buffer temporal para construir el mensaje final.
unsigned char buffer[80];

int tam =
    0;

// Agregar espacio inicial.
for (int i = 0; i < 8; i++) {
    buffer[tam++] =
        0x00;
}

// Agregar decenas cuando el puntaje sea mayor o igual a 10.
if (puntos >= 10) {
    for (int i = 0; i < 8; i++) {
        buffer[tam++] =
            digitos[decenas][i];
    }
}

// Agregar unidades.
for (int i = 0; i < 8; i++) {
    buffer[tam++] =
        digitos[unidades][i];
}

// Agregar la palabra PUNTOS.
for (int i = 0; i < 48; i++) {
    buffer[tam++] =
        PUNTOS[i];
}

// Agregar espacio final.
for (int i = 0; i < 8; i++) {
    buffer[tam++] =
        0x00;
}

// Desplazar el mensaje completo en la matriz.
for (
    int i = 0;
    i <= tam - 8;
    i++

```

```

    ) {
        for (int k = 0; k < 50; k++) {
            for (int j = 0; j < 8; j++) {
                PORTD =
                    posiciones[j];

                PORTB =
                    ~buffer[i + j];

                _delay_ms(0.05);
            }
        }

        // Apagar matriz después del mensaje.
        PORTD = 0x00;
        PORTB = 0xFF;
    }
}

```

Esta función se encarga de según un valor de puntos, poder mostrar en la matriz, el puntaje seguido de la palabra puntos, realiza un desplazamiento horizontal del mensaje.

```

int main(void) {
    // PORTD controla las filas de la matriz.
    DDRD =
        0xFF;

    // PORTB controla las columnas de la matriz.
    DDRB =
        0xFF;

    // PC0-PC2 funcionan como salidas hacia el PIC.
    // PC3-PC5 funcionan como entradas para pulsadores.
    DDRC =
        0x07;

    // Activar pull-ups internos en PC3-PC5.
    // Mantener el bus PC0-PC2 inicialmente en 000.
    PORTC =
        0x38;

    // Preparar ADC6 para leer el botón SUBIR.
    inicializar_adc();

    while (1) {
        // Mostrar bienvenida antes de cada partida.
    }
}

```

```

    estado_bienvenido();

    _delay_ms(10);

    // Permitir que el usuario seleccione el nivel.
    int nivel =
        estado_seleccion_nivel();

    // Ejecutar el Pong y guardar el resultado.
    int puntos =
        estado_juego(nivel);

    // Mostrar mensaje de victoria o puntuación final.
    if (puntos == -1) {
        estado_ganador();
    }
    else {
        estado_puntuacion(
            puntos
        );
    }
}

return 0;
}

```

Esta función se encarga de ejecutar el ciclo del juego, primero se muestra la bienvenida, luego se selecciona la dificultad y se ejecuta la función de la lógica del juego, luego se ejecuta el mensaje de ganador o el puntaje, dependiendo del valor retornado de la función estado_juego y se reinicia el ciclo

5.2 PIC16F887

```

void reproducir_tono(
    unsigned int frecuencia,
    unsigned int duracion
) {
    Sound_Play(frecuencia, duracion);
    Delay_ms(10);
}

```

Esta es la función general para reproducir sonidos, pasando como argumento, la frecuencia y la duración.

```

void sonido_pared() {
    reproducir_tono(523, 35);    // Do5
}

```

```

void sonido_paleta() {
    reproducir_tono(784, 60);    // Sol5
}

void sonido_nivel() {
    reproducir_tono(659, 70);    // Mi5
    reproducir_tono(784, 90);    // Sol5
}

```

Estas tres funciones se encargan de reproducir los sonidos cortos, cuando la pelota choca con la pared, cuando la pelota coincide con la paleta y cuando se cambia de nivel.

```

void melodia_inicio() {
    reproducir_tono(523, 80);    // Do5
    reproducir_tono(659, 80);    // Mi5
    reproducir_tono(784, 130);   // Sol5
}

void melodia_derrota() {
    reproducir_tono(392, 130);   // Sol4
    reproducir_tono(330, 160);   // Mi4
    reproducir_tono(262, 260);   // Do4
}

void melodia_victoria() {
    reproducir_tono(523, 90);     // Do5
    reproducir_tono(659, 90);     // Mi5
    reproducir_tono(784, 110);    // Sol5
    reproducir_tono(1046, 260);   // Do6
}

```

Estas tres funciones lo que hacen es reproducir melodías, es decir combinaciones secuenciales de sonidos, en este caso, la primera para anunciar la bienvenida al juego, la segunda para indicar derrota y la tercera para indicar que se ha ganado el juego.

```

void musica_bienvenida() {
    if ((PORTA & 0x07) == 7) {
        reproducir_tono(262, 140);    // Do4
    }

    if ((PORTA & 0x07) == 7) {
        Delay_ms(80);
        reproducir_tono(330, 140);    // Mi4
    }

    if ((PORTA & 0x07) == 7) {
        Delay_ms(80);
        reproducir_tono(392, 180);    // Sol4
    }
}

```

```

    }

    if ((PORTA & 0x07) == 7) {
        Delay_ms(80);
        reproducir_tono(440, 180);    // La4
    }

    if ((PORTA & 0x07) == 7) {
        Delay_ms(80);
        reproducir_tono(370, 180);    // Fa_s4
    }

    if ((PORTA & 0x07) == 7) {
        Delay_ms(80);
        reproducir_tono(311, 180);    // re_s4
    }

    Delay_ms(500);
}

```

Esta función sirve para establecer la melodía mientras se siga mostrando el mensaje de bienvenida. El PIC16F887 constantemente está evaluando el bus de datos a ver si sigue enviando el comando 7 que señala que el mensaje se sigue mostrando, la validación se hace nota a nota de la melodía de bienvenida.

```

unsigned short leer_comando() {
    return PORTA & 0x07;
}

```

Esta función me lee el bus de comunicación, en específico el comando recibido.

```

void main() {
    unsigned short comando;

    // Desactivar entradas analógicas.
    ANSEL = 0x00;
    ANSELH = 0x00;

    // Desactivar comparadores.
    C10N_bit = 0;
    C20N_bit = 0;

    // RA0-RA2 como entradas.
    TRISA = 0xFF;

    // PORTC como salida.
}

```

```
TRISC = 0x00;
```

```
PORTA = 0x00;
```

```
PORTC = 0x00;
```

En este código primero se desactivan las entradas analógicas y los comparadores para evitar errores y falsas lecturas. Además, configura el puerto A como entrada para el bus de datos, el C como salida para el buzzer y deja estos registros en 0.

```
// Speaker conectado a RC3.
```

```
Sound_Init(&PORTC, 3);
```

Se establece al pin 3 para que reproduzca el sonido

```
while (1) {
    comando = leer_comando();

    if (comando == 1) {
        melodia_inicio();

        // Esperar a que el ATmega retire el comando.
        while ((PORTA & 0x07) != 0) {
            Delay_ms(1);
        }
    }
    else if (comando == 2) {
        sonido_pared();

        while ((PORTA & 0x07) != 0) {
            Delay_ms(1);
        }
    }
    else if (comando == 3) {
        sonido_paleta();

        while ((PORTA & 0x07) != 0) {
            Delay_ms(1);
        }
    }
    else if (comando == 4) {
        sonido_nivel();

        while ((PORTA & 0x07) != 0) {
            Delay_ms(1);
        }
    }
    else if (comando == 5) {
        melodia_derrota();
    }
}
```

```

        while ((PORTA & 0x07) != 0) {
            Delay_ms(1);
        }
    }
    else if (comando == 6) {
        melodia_victoria();

        while ((PORTA & 0x07) != 0) {
            Delay_ms(1);
        }
    }
    else if (comando == 7) {
        // La melodía se repite mientras el mensaje BIENVENIDO
        // permanezca visible en la matriz.
        musica_bienvenida();
    }

    Delay_ms(1);
}
}

```

Aquí lo que hacen es establecerse condicionales que evalúan el valor del comando del bus de datos y lo comparan para poder decidir qué función de reproducción de sonido ejecutar.

6 Enlace del repositorio GitHub

<https://github.com/SQuirozVargas/JuegoPong-SistemasEmbebidos/tree/main>

7 Conclusiones

- La comunicación paralela entre microcontroladores permitió optimizar recursos, evitando tener que programar rutinas de interrupción por temporizador en el ATmega328P, lo que le permitió a este operar de manera secuencial, dedicándose de manera exclusiva a la lógica del juego y su interacción con el usuario, evitando posibles problemas asociados a funciones de retardo.
- Se comprobó que el microcontrolador ATmega328P es robusto y eficiente para procesar e implementar juegos interactivos tipo arcade que requieren interactuar con pulsadores en tiempo real, sin presentar desbordamiento ni caídas en el rendimiento de la ejecución del juego.
- El PIC16F887 demostró ser un excelente aliado en el procesamiento de datos, sirviendo como un coprocesador, encargándose de la parte sonora del sistema, permitiendo que, tras recibir una orden, poder responder con baja latencia, logrando que el juego se ejecute con fluidez, brindando una mejor experiencia al usuario.

8 Recomendaciones

- Una práctica eficiente consiste en poder aprovechar la electrónica pasiva del microcontrolador, para evitar el uso de componentes externos, reduciendo espacio y optimizando el circuito, en este caso, manipulando los registros para activar las resistencias pull_up necesarias para el manejo de botones.
- Es importante mencionar que, si se quiere implementar el circuito de manera física y no solo dejarlo en una simulación, se debe mejorar la lógica del antirebote, debido a que el elemento mecánico genera ruido y falsas lecturas.
- Dentro de la comunicación paralela entre microcontroladores se recomienda añadir una línea al bus de comunicación, de manera que se pueda validar en tiempo de ejecución si un dato ha sido recibido y no se dependa de un tiempo establecido.